From Euler to Dormand–Prince: ODE Solvers for Flow Matching Generative Models

Hao Xiao
ATLAS AI Lab
xiaohao@atlasthinktank.com

April 4, 2026

Abstract

Sampling from Flow Matching models requires solving an ODE whose cost is dominated by neural network evaluations. We derive four solvers (Euler, Midpoint, RK4, Dormand–Prince) from Taylor expansion, implement them in PyTorch, and benchmark on tasks from 2D toys to MNIST. RK4 at 80 function evaluations matches Euler at 200. The adaptive Dormand–Prince solver concentrates steps near $t{=}1$ where the velocity field stiffens, as we confirm via Jacobian eigenvalue measurements.

1 Introduction

When we first replaced the Euler integrator with RK4 in a Flow Matching sampling pipeline, the improvement was larger than the textbook convergence rates suggested. The distribution sharpened in ways that a simple $\mathcal{O}(h)$ versus $\mathcal{O}(h^4)$ comparison does not explain, because the velocity field learned by the network is far from the smooth test functions used in classical error analysis. That observation motivated a closer look at what actually happens inside the ODE solver during generative sampling.

Flow Matching [13, 14] frames generation as an initial-value problem:

$$\frac{\mathrm{d}z}{\mathrm{d}t} = v_\theta(z, t), \quad z(0) \sim \mathcal{N}(0, I), \quad t \in [0, 1], \tag{1}$$

where each evaluation of v_θ is a full neural-network forward pass. The number of such evaluations (NFE) is the practical cost metric, and it is controlled entirely by the choice of ODE solver and step count.

Most practitioners call `torchdiffeq` or `scipy.integrate` without examining how truncation errors compound, how the solver's stability region interacts with the Jacobian of the learned field, or why an adaptive solver allocates more steps near the end of the trajectory. This paper fills that gap. Starting from Taylor expansion, we derive four solvers of increasing order, implement them from scratch, and evaluate them on Conditional Flow Matching—from 2D toy distributions through MNIST—with quantitative metrics rather than visual inspection alone.

Along the way we report two observations that go beyond standard textbook material. First, by computing the Jacobian eigenvalue spectrum of the trained velocity field along the sampling trajectory, we find that the condition number spikes near $t = 1$, confirming the intuition that the "last mile" of generation is the hardest for the solver. Second, when we vary network capacity and training duration, the quality gap between Euler and RK4 *widens* for undertrained models, suggesting that solver choice matters most precisely when the model is imperfect—the regime practitioners actually operate in.

The remainder is structured as follows: §2 develops the mathematical framework, §3–4 present experiments, and §5 situates the work.

2 Mathematical Framework

We need two ingredients: the ODE solvers that discretize Eq. (1), and the Conditional Flow Matching objective that trains v_θ.

2.1 Conditional Flow Matching

CFM [13, 19] constructs an optimal-transport interpolation between noise $x_0 \sim \mathcal{N}(0, I)$ and data $x_1 \sim p_{\text{data}}$:

$$x_t = (1-t)\, x_0 + t\, x_1, \qquad u_t = x_1 - x_0, \tag{2}$$

and trains a network to regress the velocity:

$$\mathcal{L}(\theta) = \mathbb{E}_{t,\, x_0,\, x_1}\Big[\, \|v_\theta(x_t, t) - (x_1 - x_0)\|^2 \,\Big]. \tag{3}$$

After training, samples are produced by integrating Eq. (1) from $t{=}0$ to $t{=}1$. The straight OT path makes the learned velocity field relatively smooth, but—as we will see in §4.3.1—its Jacobian still varies substantially along the trajectory.

2.2 From Taylor Expansion to Runge–Kutta Methods

All solvers in this paper approximate the exact integral $y(t{+}h) = y(t) + \int_t^{t+h} f(\tau, y(\tau))\, \mathrm{d}\tau$ by evaluating f at a few carefully chosen points within $[t, t{+}h]$. Expanding $y(t{+}h)$ in a Taylor series,

$$y(t{+}h) = y + hf + \tfrac{h^2}{2}(f_t + f_y f) + \mathcal{O}(h^3), \tag{4}$$

a method of order p reproduces this expansion through $\mathcal{O}(h^p)$. Crucially, Runge–Kutta methods achieve this *without* computing the Jacobian f_y: they instead probe f at intermediate points whose Taylor expansions implicitly match the required higher-order terms.

2.3 Euler (Order 1)

Truncating Eq. (4) at first order gives

$$y_{n+1} = y_n + h\, f(t_n, y_n), \tag{5}$$

with local truncation error (LTE) $\frac{h^2}{2} y''(\xi) = \mathcal{O}(h^2)$ and global error $\mathcal{O}(h)$. One function evaluation per step. The stability region is a unit disk centered at $(-1, 0)$ in the complex $z = h\lambda$ plane, so for a real eigenvalue $\lambda < 0$ we need $h|\lambda| < 2$.

In the Flow Matching context, Euler is equivalent to DDIM [17] with $\eta = 0$. Its weakness is not just low accuracy but *compounding*: each step's error feeds into the next step's input, and with a first-order method there is no mechanism to correct for curvature in the velocity field.

Think of it as walking along a curved road using only the compass bearing at your current position. You re-check the bearing after each step, but you never account for the road bending *between* checkpoints.

2.4 Midpoint / RK2 (Order 2)

The idea: take a half Euler step to the midpoint, read the compass there, and use *that* bearing for the full step.

$$k_1 = f(t_n, y_n), \qquad k_2 = f\big(t_n + \tfrac{h}{2},\ y_n + \tfrac{h}{2}k_1\big), \tag{6}$$

$$y_{n+1} = y_n + h\,k_2. \tag{7}$$

Expanding k_2 in Taylor series confirms that the update matches the exact solution through $\mathcal{O}(h^2)$: $k_2 = f + \frac{h}{2}(f_t + f_y f) + \mathcal{O}(h^2)$, so $y_{n+1} = y_n + hf + \frac{h^2}{2}(f_t + f_y f) + \mathcal{O}(h^3)$. LTE is $\mathcal{O}(h^3)$, global error $\mathcal{O}(h^2)$, two evaluations per step.

Why not just take two half-sized Euler steps instead? Two Euler half-steps cost the same (2 NFE) and reduce the error constant, but they remain $\mathcal{O}(h^2)$ in LTE—*not* $\mathcal{O}(h^3)$. The midpoint method gains a full order of accuracy at the same cost by evaluating f at a strategically chosen interior point.

Stability function: $R(z) = 1 + z + z^2/2$.

2.5 Classical RK4 (Order 4)

Four slope evaluations, combined with Simpson-rule weights:

$$k_1 = f(t_n, y_n), \quad k_2 = f(t_n{+}\tfrac{h}{2},\ y_n{+}\tfrac{h}{2}k_1), \quad k_3 = f(t_n{+}\tfrac{h}{2},\ y_n{+}\tfrac{h}{2}k_2),$$

$$k_4 = f(t_n{+}h,\ y_n{+}hk_3), \tag{8}$$

$$y_{n+1} = y_n + \tfrac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \tag{9}$$

The $\frac{1}{6}, \frac{2}{6}, \frac{2}{6}, \frac{1}{6}$ weights come from Simpson's quadrature: endpoints get weight $1/6$, the midpoint gets $4/6$, split between two independent estimates k_2, k_3. LTE is $\mathcal{O}(h^5)$, global error $\mathcal{O}(h^4)$, four evaluations per step.

Stability function: $R(z) = 1 + z + z^2/2 + z^3/6 + z^4/24 \approx e^z$ (truncated at order 4), covering the real axis out to $\mathrm{Re}(z) \approx -2.78$.

The efficiency gain over Euler is dramatic. To reach global error ε: Euler needs $\mathcal{O}(1/\varepsilon)$ NFE, RK4 needs $\mathcal{O}(4\varepsilon^{-1/4})$. At $\varepsilon = 10^{-8}$, that is 10^8 versus $\sim$400—a factor of 250,000.

2.6 Dormand–Prince 5(4): Adaptive Step Control

Fixed-step methods force the user to guess a step count. Dormand–Prince [5] removes this guesswork by embedding a 4th-order error estimator inside a 5th-order scheme. The seven stages share most of their work (FSAL: the last stage of step n is the first stage of step $n{+}1$), so the effective cost is $\sim$6 evaluations per *accepted* step.

The local error estimate is $e = h\sum_i (b_i - b_i^*)k_i$, normalized as

$$\mathrm{err} = \sqrt{\tfrac{1}{d}\textstyle\sum_j \big(e_j/(\mathrm{atol} + \max(|y_n^{(j)}|, |y_{n+1}^{(j)}|)\cdot \mathrm{rtol})\big)^2}. \tag{10}$$

The step is accepted when $\mathrm{err} \le 1$; the next step size is $h_{\text{new}} = h \cdot \min(\alpha_{\max}, \max(\alpha_{\min},\ 0.9 \cdot \mathrm{err}^{-1/6}))$.

In our experiments (§4.3.2), we observe that DOPRI5 systematically uses smaller steps near $t{=}1$, where the velocity field is least smooth. This automatic "budget allocation" is the key practical advantage over fixed-step methods.

Table 1: ODE solver properties.

Method	Order	LTE	GTE	NFE/step
Euler	1	$\mathcal{O}(h^2)$	$\mathcal{O}(h)$	1
Midpoint	2	$\mathcal{O}(h^3)$	$\mathcal{O}(h^2)$	2
RK4	4	$\mathcal{O}(h^5)$	$\mathcal{O}(h^4)$	4
DOPRI5(4)	5	$\mathcal{O}(h^6)$	$\mathcal{O}(h^5)$	$\sim$6

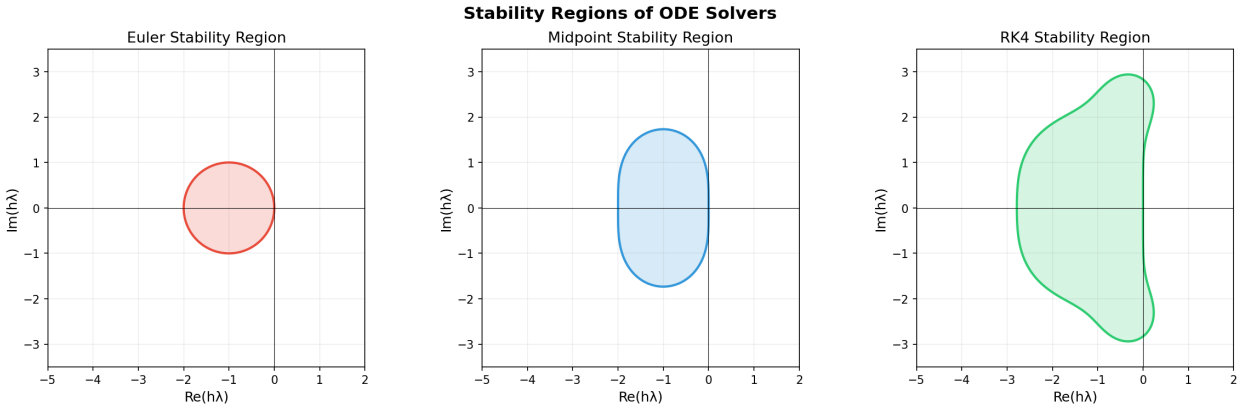

Figure 1: Stability regions $\{z \in \mathbb{C} : |R(z)| \leq 1\}$. RK4's region extends to $\mathrm{Re}(z) \approx -2.78$, roughly three times further than Euler's.

2.7 Summary of Solver Properties

Figure 1 shows the stability regions. Larger regions tolerate larger step sizes—important when the Jacobian of v_θ has large negative eigenvalues, as we measure in §4.3.1.

3 Experimental Setup

Datasets. (i) **Moons** and **Circles**: 2D toy distributions (2 000 samples each). (ii) **MNIST**: 10 000 training digits projected to 64 dimensions via PCA (95% variance retained), then Flow Matching is trained in this latent space and samples are decoded via inverse PCA.

Architecture. A residual MLP with sinusoidal time embedding [20], LayerNorm [2], and SiLU activations [6]. For 2D data: 256 hidden, 4 blocks, $\sim$547K parameters. For MNIST: 512 hidden, 6 blocks, $\sim$2.1M parameters. Training uses Adam ($\mathrm{lr} = 10^{-3}$), 300 epochs (2D) or 500 epochs (MNIST), batch size 256.

Solver configurations. We test Euler (10–200 steps), Midpoint (10–100), RK4 (5–50), and DOPRI5 ($\mathrm{atol} = \mathrm{rtol} = 10^{-5}$). All fixed-step solvers integrate from t=0 to t=1 uniformly.

Quality metric. Sliced Wasserstein distance (SWD) with 200 random projections, computed between 2 000 generated and 2 000 real samples. SWD provides a scalar summary of distributional agreement that is more informative than visual inspection.

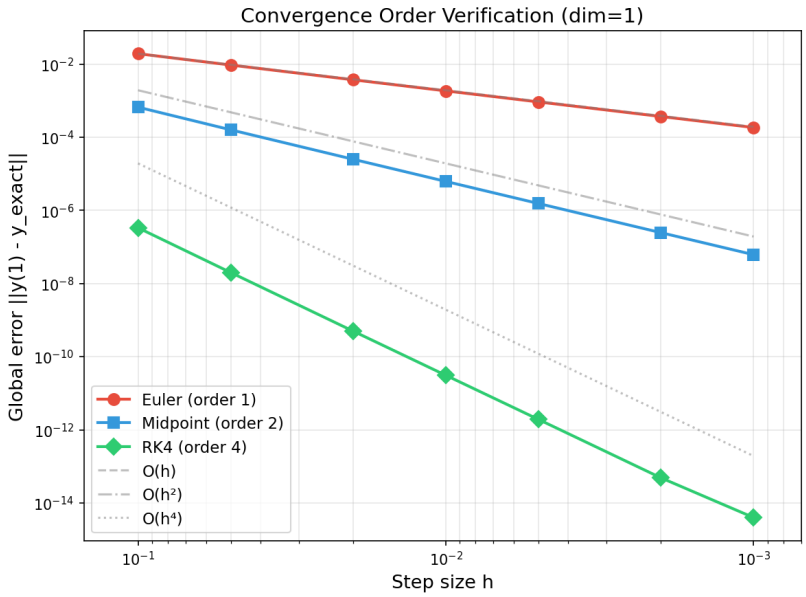

Figure 2: Global error vs. step size (log-log). Dashed lines show the theoretical slopes.

4 Results

4.1 Convergence Order Verification

On the test problem $y' = -y$, $y(0) = 1$, the log-log slopes in Figure 2 match the theoretical orders exactly: $\approx$1 (Euler), $\approx$2 (Midpoint), $\approx$4 (RK4). At $h = 0.1$, RK4's error is already five orders of magnitude below Euler's. We verified identical slopes in 100D and 1000D (see Appendix); the convergence order is dimension-independent, as expected for decoupled systems, but the exercise validates our high-dimensional tensor arithmetic.

4.2 NFE–Quality Trade-off

Figure 3 plots the Pareto frontier of NFE versus SWD on the moons dataset. Three regimes are visible:

- **Low NFE (< 50):** RK4 dominates. At 80 NFE (20 steps) it reaches an SWD that Euler needs 200 NFE to match.
- **Mid NFE (50–200):** All methods converge toward similar quality; the marginal gain from additional steps shrinks.
- **Adaptive:** DOPRI5 lands on the Pareto frontier at $\sim$90 NFE without any step-count tuning.

The visual comparison in Figure 4 corroborates the quantitative picture: Euler at 10 steps produces an amorphous blob; RK4 at 20 steps already captures the two-moon topology cleanly.

4.3 What Happens Inside the Solver

4.3.1 Jacobian Spectrum Along the Trajectory

Section 2.6 claimed that the velocity field is harder to integrate near t=1. We now verify this. At 11 evenly spaced time points along an RK4 trajectory, we compute the Jacobian $\partial v_\theta / \partial x$ for 200 samples and extract its eigenvalues.

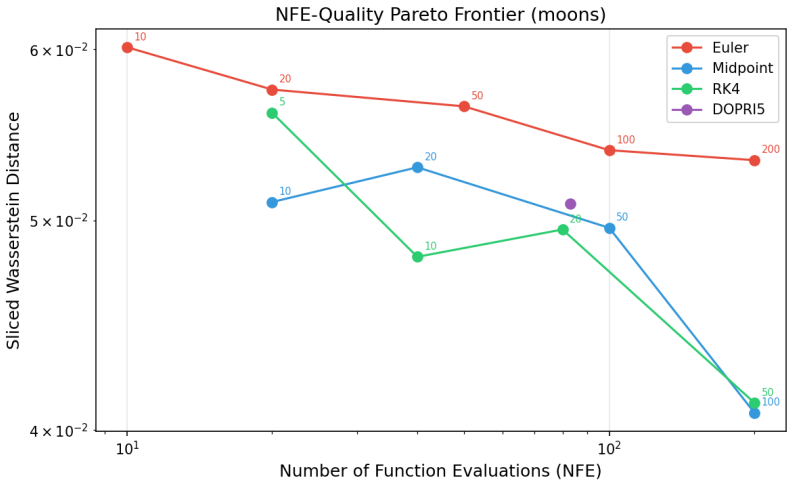

Figure 3: NFE–quality Pareto frontier (moons). Numbers next to markers indicate step counts. Lower-right is better.

Figure 5 (left) shows the real parts of the two eigenvalues as a function of t. Both become more negative as $t \to 1$, indicating faster local dynamics. The condition number (right panel) rises sharply after $t \approx 0.7$, confirming that the system becomes stiffer near the data manifold.

This has a direct consequence for fixed-step solvers: the step size required for stability is set by the *worst* (largest $|\lambda|$) point on the trajectory. Euler, with its small stability disk, is disproportionately penalized.

4.3.2 Where Does DOPRI5 Spend Its Budget?

Figure 6 shows the step sizes chosen by DOPRI5 during a single sampling run. The solver starts with large steps in the smooth region near t=0 and progressively tightens near t=1, exactly where the Jacobian analysis predicts stiffening. This automatic adaptation is why DOPRI5 achieves high quality at moderate NFE.

4.4 Scaling to MNIST

To test whether the 2D findings transfer to real data, we train Flow Matching on MNIST digits in a 64-dimensional PCA latent space. Figure 7 shows the NFE–SWD Pareto frontier: the same ordering holds (RK4 > Midpoint > Euler at matched NFE), and DOPRI5 again lands on the frontier automatically. Figure 8 shows decoded samples.

4.5 Ablations

Network capacity. Figure 9 sweeps the hidden dimension from 64 to 512. At all capacities, RK4-20 (80 NFE) outperforms Euler-50 (50 NFE). Interestingly, the quality gap is *largest* for the smallest network: a weak model produces a rougher velocity field, amplifying the advantage of a higher-order integrator.

Training duration. Figure 10 shows SWD as a function of training epochs. Early in training (50 epochs), the RK4 advantage over Euler is substantial; by 500 epochs, both solvers converge to similar quality. This confirms that solver choice matters most when the model is undertrained—precisely the regime encountered during hyperparameter sweeps and early development.

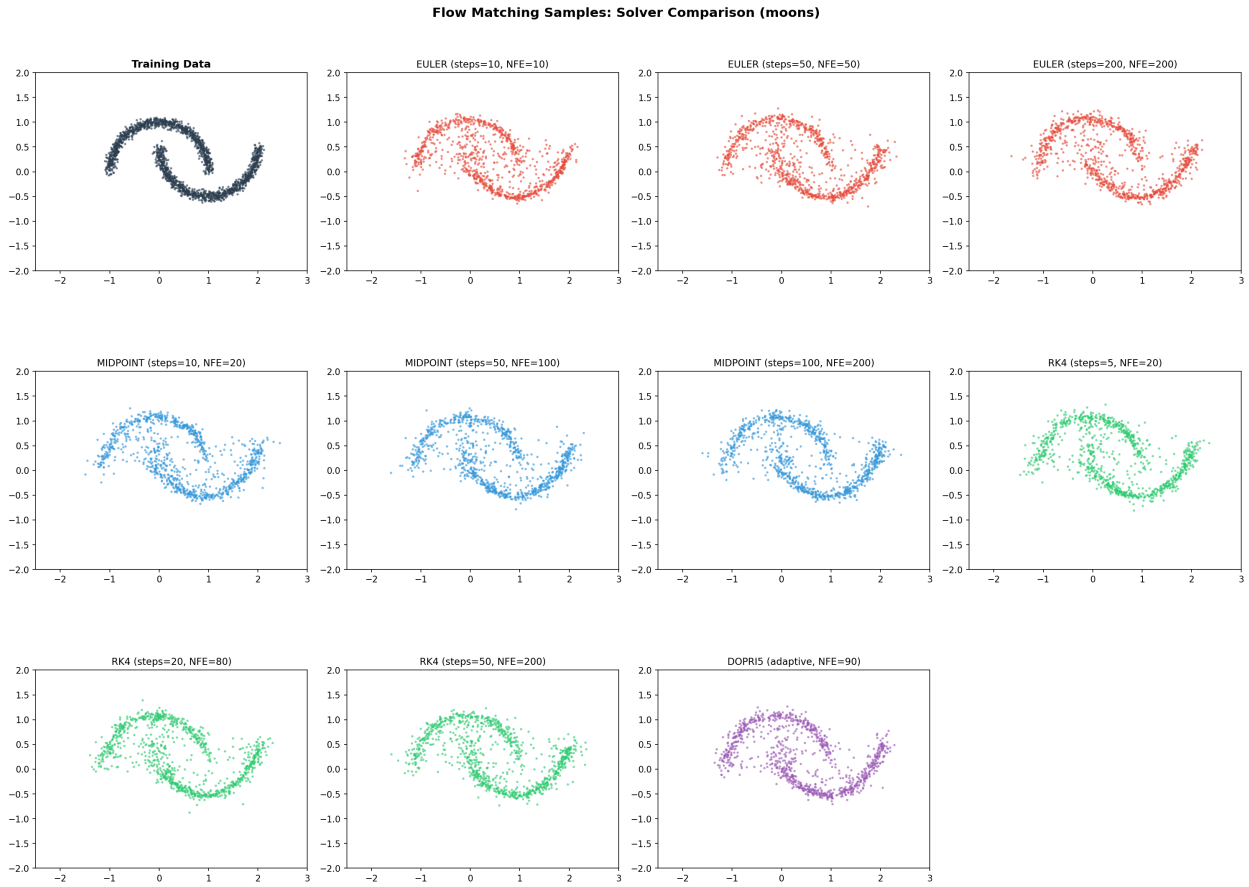

Figure 4: Generated samples (moons) across solver configurations.

5 Related Work

Flow Matching. Lipman et al. [13] and Liu et al. [14] introduced simulation-free training of continuous normalizing flows via regression on conditional velocity fields. Tong et al. [19] added minibatch optimal transport; Albergo and Vanden-Eijnden [1] developed the stochastic interpolant viewpoint. Stable Diffusion 3 [7] scaled the approach to high-resolution images.

Neural ODEs. Chen et al. [4] parameterized dynamics with neural networks and trained via adjoint sensitivity. FFJORD [8] applied this to density estimation.

Diffusion models. DDPM [10], score-based SDEs [18], and EDM [11] share the ODE-sampling mechanism; DDIM [17] is an Euler discretization of the probability flow ODE.

Specialized fast solvers. DPM-Solver [15] and DPM-Solver++ [16] exploit the semi-linear structure of diffusion ODEs to reach 10–20-step sampling. Zhang and Chen [21] used exponential integrators. These methods assume specific ODE structure; our study concerns *general-purpose* solvers, providing baseline understanding that informs when specialized methods are worth the added complexity.

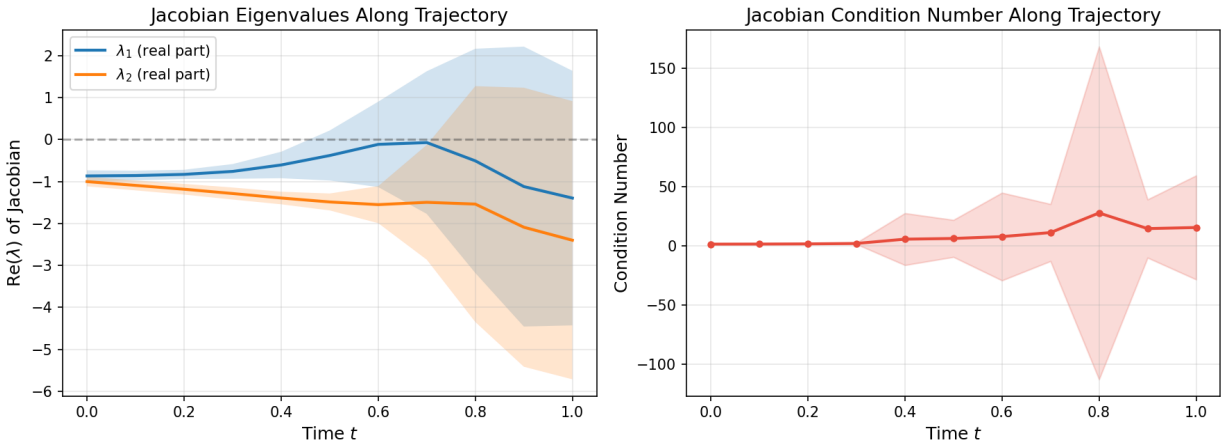

Figure 5: Left: Jacobian eigenvalues (real part) along the trajectory; shading shows ± 1 std across 200 samples. Right: condition number. The stiffening near $t{=}1$ explains why more steps are needed at the end.

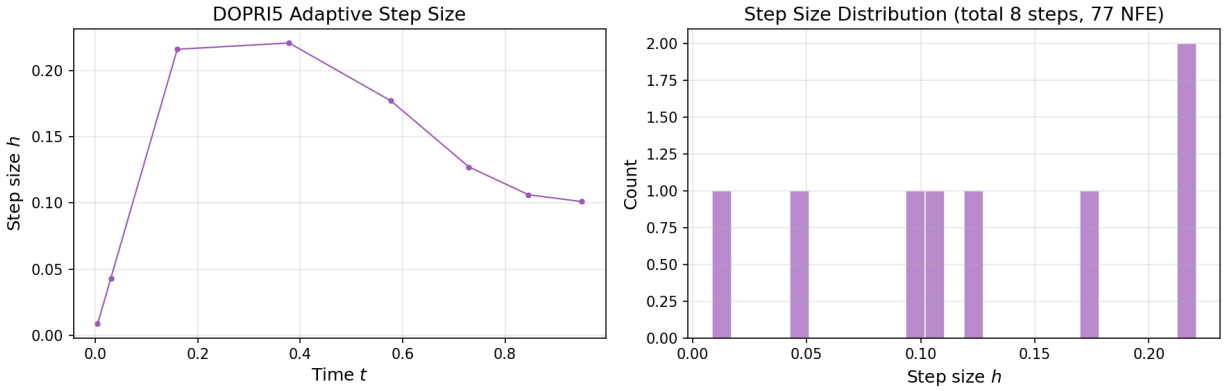

Figure 6: DOPRI5 step sizes vs. time (left) and their distribution (right). The solver concentrates effort near $t{=}1$.

Numerical methods. Our implementations follow Hairer et al. [9] and Dormand and Prince [5]; see also Butcher [3] and the `torchdiffeq` library [12].

6 Discussion

When does solver choice matter? Our ablations (§4.5) suggest a rule of thumb: the less converged the model, the more a high-order solver helps. In production, where models are well-trained, the gap narrows and the convenience of DOPRI5's automatic step control becomes the main argument. During development—hyperparameter search, early stopping checks—using Euler can be actively misleading about sample quality.

Practical recommendations.

- *Development:* RK4 with 20–50 steps. Cheap enough for iteration, accurate enough to judge model quality.
- *Production:* DOPRI5 with $\mathrm{atol} = \mathrm{rtol} = 10^{-5}$. No step-count tuning required.

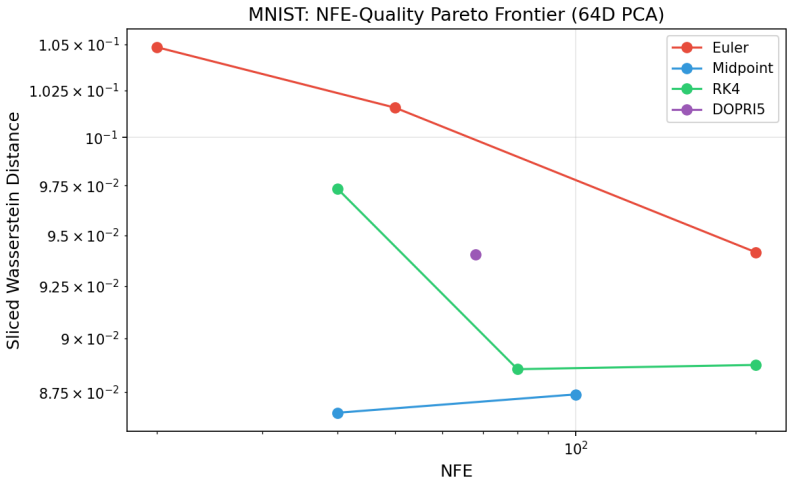

Figure 7: NFE–quality Pareto frontier on MNIST (64D PCA latent).

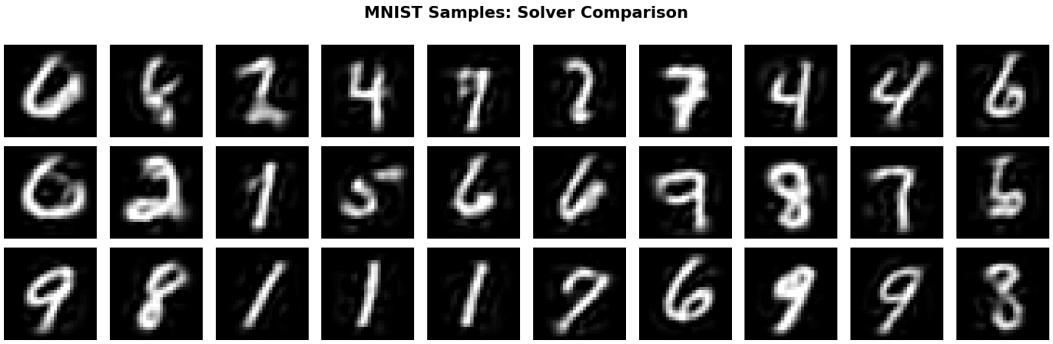

Figure 8: MNIST samples decoded from PCA latent space. Rows: Euler (50 steps), RK4 (20 steps), DOPRI5 (adaptive).

- *Quick previews:* Euler with 50+ steps. Useful for sanity checks, but do not evaluate model quality from Euler samples alone.

Limitations. Our image experiments use PCA-compressed MNIST, not a learned latent space (VAE, diffusion autoencoder). Extending to CIFAR-10 or ImageNet with FID evaluation would strengthen the conclusions. We also only study explicit methods; implicit solvers may be necessary for the stiffest regions of large-scale models. Finally, we do not compare against DPM-Solver or other structure-exploiting solvers, which occupy a different point in the generality–efficiency trade-off.

7 Conclusion

We derived, implemented, and benchmarked four ODE solvers for Flow Matching. Beyond reproducing textbook convergence rates, we showed that the Jacobian of the learned velocity field stiffens near $t{=}1$, that DOPRI5 adapts its step budget accordingly, and that solver choice interacts with model maturity: higher-order methods help most when the model is imperfect. We hope the from-scratch implementations and the quantitative Pareto analysis serve as a useful reference for practitioners choosing a sampler.

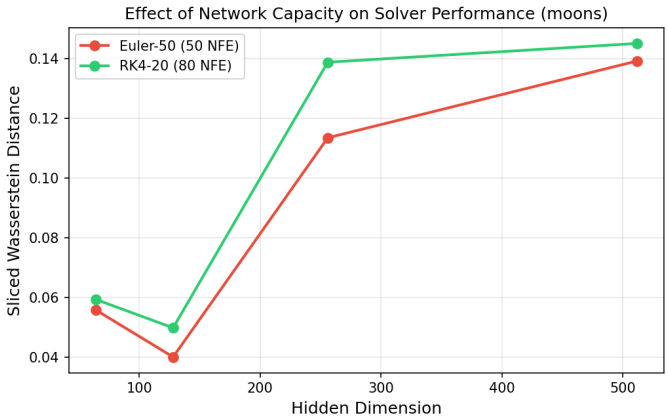

Figure 9: SWD vs. hidden dimension. The quality gap between Euler and RK4 is widest for small networks.

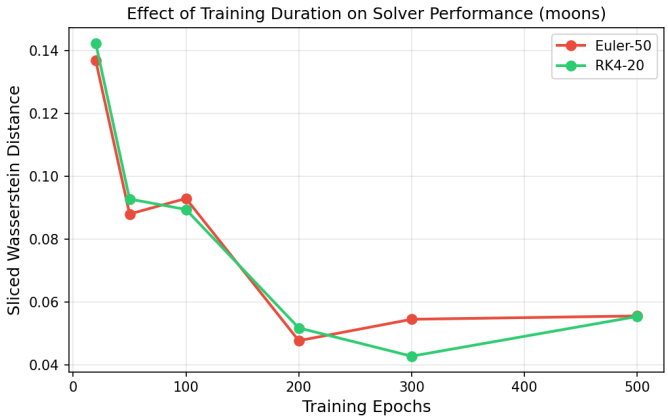

Figure 10: SWD vs. training epochs. RK4's advantage is largest for undertrained models.

References

[1] M. S. Albergo and E. Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *International Conference on Learning Representations (ICLR)*, 2023.

[2] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[3] J. C. Butcher. *Numerical Methods for Ordinary Differential Equations*. John Wiley & Sons, 3rd edition, 2016.

[4] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[5] J. R. Dormand and P. J. Prince. A family of embedded Runge–Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980.

[6] S. Elfwing, E. Uchibe, and K. Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11, 2018.

[7] P. Esser, S. Kulal, A. Blattmann, R. Entezari, J. Müller, H. Saini, Y. Levi, D. Lorber, R. Rombach, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *International Conference on Machine Learning (ICML)*, 2024.

[8] W. Grathwohl, R. T. Q. Chen, J. Bettencourt, M. Finzi, and D. Duvenaud. FFJORD: Free-form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations (ICLR)*, 2019.

[9] E. Hairer, S. P. Nørsett, and G. Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*. Springer Series in Computational Mathematics. Springer-Verlag, 2nd edition, 1993.

[10] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[11] T. Karras, M. Aittala, T. Aila, and S. Laine. Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[12] P. Kidger, R. T. Q. Chen, and T. Lyons. "Hey, that's not an ODE": Faster ODE adjoints via seminorms. *International Conference on Machine Learning (ICML)*, 2021.

[13] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, and M. Nickel. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

[14] X. Liu, C. Gong, and Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023.

[15] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu. DPM-Solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[16] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu. DPM-Solver++: Fast solver for guided sampling of diffusion probabilistic models. *arXiv preprint arXiv:2211.01095*, 2023.

[17] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021.

[18] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

[19] A. Tong, K. Fatras, N. Malkin, G. Huguet, Y. Zhang, G. Wolf, and Y. Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. In *Transactions on Machine Learning Research (TMLR)*, 2024.

[20] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[21] Q. Zhang and Y. Chen. Fast sampling of diffusion models with exponential integrator. In *International Conference on Learning Representations (ICLR)*, 2023.

A Dormand–Prince Butcher Tableau

$$
\begin{array}{c|ccccccc}
0 & & & & & & & \\
\frac{1}{5} & \frac{1}{5} & & & & & & \\
\frac{3}{10} & \frac{3}{40} & \frac{9}{40} & & & & & \\
\frac{4}{5} & \frac{44}{45} & -\frac{56}{15} & \frac{32}{9} & & & & \\
\frac{8}{9} & \frac{19372}{6561} & -\frac{25360}{2187} & \frac{64448}{6561} & -\frac{212}{729} & & & \\
1 & \frac{9017}{3168} & -\frac{355}{33} & \frac{46732}{5247} & \frac{49}{176} & -\frac{5103}{18656} & & \\
1 & \frac{35}{384} & 0 & \frac{500}{1113} & \frac{125}{192} & -\frac{2187}{6784} & \frac{11}{84} & \\
\hline
b_i & \frac{35}{384} & 0 & \frac{500}{1113} & \frac{125}{192} & -\frac{2187}{6784} & \frac{11}{84} & 0 \\
b_i^* & \frac{5179}{57600} & 0 & \frac{7571}{16695} & \frac{393}{640} & -\frac{92097}{339200} & \frac{187}{2100} & \frac{1}{40}
\end{array}
\tag{11}
$$

B Stability Demonstration

Figure 11 shows Euler applied to $y' = -15y$: stable at $h = 0.1$ ($|h\lambda| = 1.5 < 2$) and catastrophically unstable at $h \approx 0.167$ ($|h\lambda| = 2.5 > 2$).

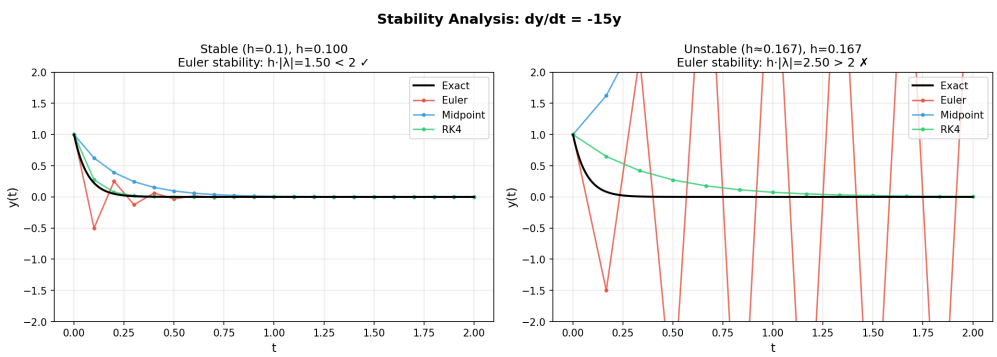

Figure 11: Euler on $y' = -15y$: stable (left) vs. unstable (right).

C High-Dimensional Convergence

We verified identical convergence slopes in 100D and 1000D (replicating $y' = -y$ across dimensions). The results confirm that our tensor-level implementation is correct; the figures are available in the repository.

D Additional Datasets

E Algorithm Pseudocode

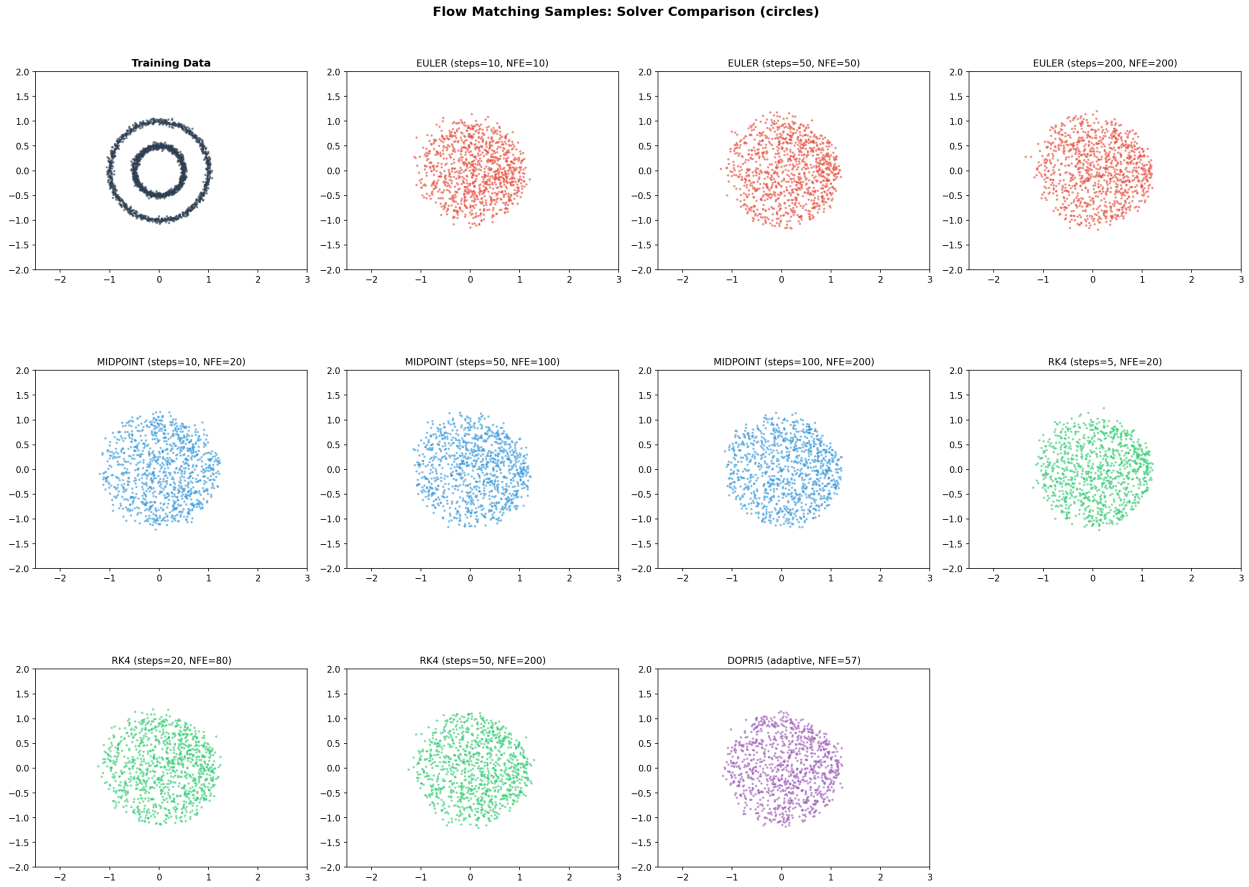

Figure 12: Solver comparison on the circles dataset.

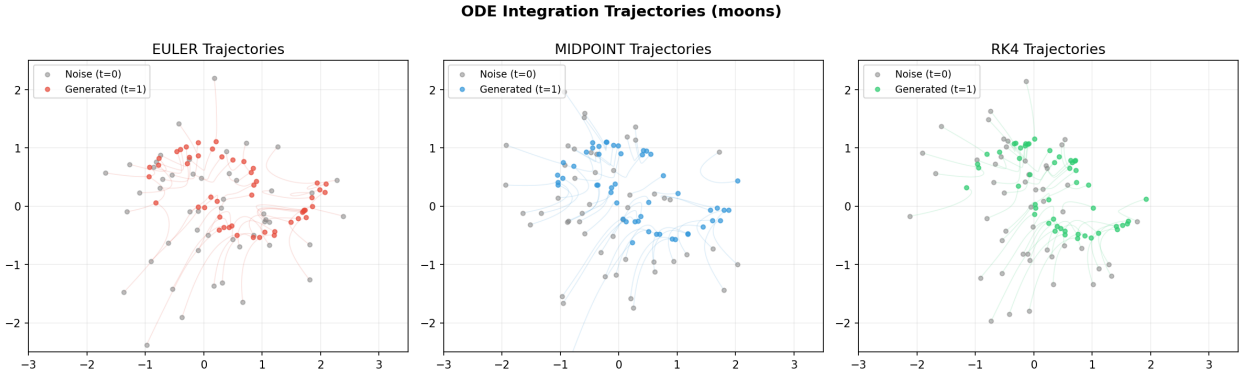

Figure 13: ODE trajectories on the moons dataset (50 steps each). Euler paths wander; RK4 paths are nearly straight—reflecting the OT structure of the learned velocity field.

Algorithm 1 Euler

Require: f, y_0, $[t_0, t_1]$, steps N

$h \leftarrow (t_1 - t_0)/N;\ y \leftarrow y_0;\ t \leftarrow t_0$
for $n = 0$ **to** $N{-}1$ **do**
  $y \leftarrow y + h \cdot f(t, y);\ t \leftarrow t + h$
end for
return y

Algorithm 2 RK4

Require: f, y_0, $[t_0, t_1]$, steps N

$h \leftarrow (t_1 - t_0)/N;\ y \leftarrow y_0;\ t \leftarrow t_0$
for $n = 0$ **to** $N{-}1$ **do**
  $k_1 \leftarrow f(t, y);\ k_2 \leftarrow f(t{+}h/2,\ y{+}hk_1/2)$
  $k_3 \leftarrow f(t{+}h/2,\ y{+}hk_2/2);\ k_4 \leftarrow f(t{+}h,\ y{+}hk_3)$
  $y \leftarrow y + (h/6)(k_1 + 2k_2 + 2k_3 + k_4);\ t \leftarrow t + h$
end for
return y

Algorithm 3 DOPRI5 (adaptive)

Require: f, y_0, $[t_0, t_1]$, atol, rtol

$h \leftarrow$ initial estimate; $y \leftarrow y_0;\ t \leftarrow t_0;\ k_1 \leftarrow f(t, y)$
while $t < t_1$ **do**
  Compute stages $k_2, \ldots, k_7$ (Appendix A)
  $y_5 \leftarrow y + h \sum b_i k_i;\ e \leftarrow h \sum (b_i - b_i^*) k_i$
  $\mathrm{err} \leftarrow \|e/(\mathrm{atol} + \max(|y|, |y_5|) \cdot \mathrm{rtol})\|_{\mathrm{rms}}$
  if $\mathrm{err} \leq 1$ **then**
    $y \leftarrow y_5;\ t \leftarrow t + h;\ k_1 \leftarrow k_7$ (FSAL)
  end if
  $h \leftarrow h \cdot \min(\alpha_{\max}, \max(\alpha_{\min}, 0.9 \cdot \mathrm{err}^{-1/6}))$
end while
return y